BÁO CÁO CHUYÊN ĐỀ & THUYẾT TRÌNH MICROSERVICES

GIAO TIẾP SERVICE-TO-SERVICE TRONG HỆ THỐNG ĐẶT SÂN THỂ THAO

Khảo sát phân tích kiến trúc, triển khai đồng bộ REST API (Spring Boot 3 & FastAPI Python), cấu hình Timeout, xử lý lỗi mạng & tích hợp Message Broker RabbitMQ bất đồng bộ.

DỰ ÁN ÁP DỤNG	CÔNG NGHỆ CHÍNH	QUY MÔ LAB	MỤC TIÊU ĐẠT ĐƯỢC
Alobo Court Platform (`D:\hdvackend`)	Spring Boot 3, FastAPI, RabbitMQ	5 Chuyên đề (150 Phút)	Resilient & Event-Driven Services

Lộ Trình Thuyết Trình & Kiến Trúc Tổng Thể

Nắm vững các khía cạnh từ phân tích, lập trình đến vận hành khả năng chịu lỗi

Bài 1: Khảo sát & Phân tích Giao tiếp Service-to-Service — 20 PHÚT

Liệt kê 5 service, phân tích dependency, định nghĩa hợp đồng REST, sơ đồ luồng dữ liệu.

Bài 2: Triển khai Giao tiếp Bằng Spring Boot 3 — 35 PHÚT

Xây dựng field-service (Provider) & booking-service (Consumer với RestClient, 200/404 handling).

Bài 3: Tiêu thụ API bằng Framework khác (FastAPI Python) — 35 PHÚT

Viết lại consumer bằng FastAPI + HTTPX; lập bảng so sánh 5 tiêu chí với Spring Boot.

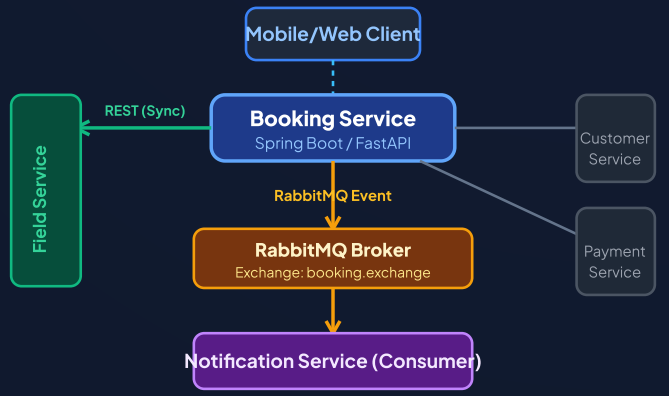
Bài 4: Xử lý Lỗi, Timeout & Giao tiếp Bất đồng bộ RabbitMQ — 40 PHÚT

Bắt lỗi mạng Connection Refused, cấu hình timeout 3s (504), Producer & Consumer RabbitMQ.

Bài 5: Thử thách Mở rộng & Giải đáp 5 Exit Tickets — 20 PHÚT

Loại bỏ hard-code qua file cấu hình, cơ chế Retry/Fallback, giải đáp toàn diện 5 câu hỏi lý thuyết.

KIẾN TRÚC TỔNG THỂ MICROSERVICES



Đồng bộ với kho mã nguồn dự án: `NestJS / Express` & `python_backend (FastAPI)`.

1.1 Khảo Sát Danh Sách 5 Microservices Cốt Lõi

Phân định ranh giới nghiệp vụ (Bounded Contexts) chuẩn Domain-Driven Design (DDD)

Tên Microservice	Vai Trò / Phân Loại	Trách Nhiệm Nghiệp Vụ Cốt Lõi	Port Chuẩn	Giao Thức
Field Service	DATA PROVIDER	Quản lý danh mục sân (catalog/yards), môn thể thao (bóng đá, pickleball, cầu lông), bảng giá giờ cao điểm/thấp điểm, kiểm tra trạng thái hoạt động của sân (AVAILABLE / MAINTENANCE).	:8081	HTTP / REST
Booking Service	CORE CONSUMER	Xử lý đặt lịch sân, khóa khung giờ (Hold slot), xác thực sự tồn tại và khả dụng của sân từ Field Service, lưu trữ đơn đặt sân và phát sự kiện hoàn tất.	:8080	HTTP & AMQP
Customer Service	IDENTITY PROVIDER	Quản lý thông tin định danh khách hàng (Profile, E.164 Phone, OTP Auth, phân quyền role branch_owner/customer), xác minh tư cách người đặt sân.	:8082	HTTP / REST
Payment Service	BILLING SERVICE	Tạo mã thanh toán (Payment Intent), xác nhận giao dịch cọc/toàn phần qua Sandbox/Ngân hàng, sinh mã hóa đơn và đối soát doanh thu chi nhánh.	:8083	HTTP / Webhook
Notification Service	EVENT CONSUMER	Lắng nghe sự kiện từ hàng đợi RabbitMQ, gửi thông báo đẩy (FCM Push), tin nhắn SMS/Zalo OTP, Email xác nhận lịch đặt sân cho cả khách hàng và chủ sân.	:8084	RabbitMQ (AMQP)

MỐI LIÊN HỆ VỚI DỰ ÁN THỰC TẾ D:\HDVACKEND

Trong repo `D:\hdvackend`, các module này được hiện thực hóa ở cả hai phiên bản: **NodeJS/TypeScript** (NestJS/Express) và **Python Backend** (`python_backend/alobo_api` với FastAPI). Điều này mang lại sự linh hoạt tối đa khi triển khai đa nền tảng (Polyglot Microservices).

1.2 Phân Tích Quan Hệ Phụ Thuộc (Dependency) & API Contract

Bảo đảm tính toàn vẹn nghiệp vụ (Business Integrity) trước khi ghi nhận đơn đặt sân

TẠI SAO PHẢI XÁC THỰC SÂN QUA FIELD-SERVICE?

- **Nguyên tắc Single Source of Truth:** Thông tin sân bãi, giá thuê theo giờ và trạng thái bảo trì chỉ do *Field Service* toàn quyền sở hữu và quản lý trong cơ sở dữ liệu riêng.
- **Ngăn chặn Đặt sân Hư hỏng/Đóng cửa:** Sân có thể đang ở trạng thái `MAINTENANCE` hoặc `CLOSED`. Nếu không kiểm tra, hệ thống sẽ xác nhận lịch cho một sân không sử dụng được.
- **Tránh sai lệch Doanh thu (Pricing Integrity):** Giá thuê sân thực tế (`pricePerHour`) cần được lấy chính xác tại thời điểm đặt để tính tổng cọc, không lưu cứng ở client.
- **Phụ thuộc Chiều Một (One-way Dependency):** Booking Service phụ thuộc vào Field Service; Field Service hoàn toàn không biết và không phụ thuộc vào Booking Service.

HỢP ĐỒNG API: GET /API/FIELDS/{ID}

`GET` **`/api/fields/{id}`**

Phản hồi thành công (200 OK):

```
// Content-Type: application/json
{
  "id": "f-101",
  "name": "Sân bóng đá cỏ nhân tạo 7 người - Sân A",
  "sportType": "FOOTBALL",
  "pricePerHour": 350000.0,
  "status": "AVAILABLE"
}
```

Phản hồi khi không tìm thấy sân (404 Not Found):

```
{
  "timestamp": "2026-10-05T14:00:00Z",
  "status": 404,
  "error": "Not Found",
  "message": "Sân thể thao với mã 'f-999' không tồn tại!"
}
```

1.3 Sơ Đồ Luồng Dữ Liệu Đồng Bộ HTTP/REST

Mô hình tuần tự (Sequence Flow) giữa Booking Service và Field Service

CÁC BƯỚC THỰC HIỆN TUẦN TỰ

1. **Client gửi yêu cầu đặt sân:** Gửi request `POST /api/bookings` với dữ liệu: `customerId, fieldId, bookingDate, startTime, endTime`.
2. **Booking Service chặn luồng:** Tạm ngưng việc ghi cơ sở dữ liệu để thực hiện cuộc gọi đồng bộ `GET /api/fields/{fieldId}` sang Field Service.
3. **Field Service truy vấn Catalog DB:**
   - Nếu sân tồn tại: trả `200 OK` kèm JSON thông tin sân.
   - Nếu không có: trả `404 Not Found`.
4. **Phân nhánh xử lý:**
   - *Hợp lệ (200 OK):* Tạo bản ghi Booking, trả về `201 Created` cho Client.
   - *Không tìm thấy (404):* Hủy giao dịch, trả về `404 Error` cho Client.

Đặc điểm Đồng bộ: Luồng xử lý của Booking Service phải chờ (blocking) cho đến khi Field Service hoàn thành và trả phản hồi về mạng.



2.1 Xây Dựng Provider (field-service) Bằng Spring Boot 3

Triển khai REST Endpoint GET /api/fields/{id} chuẩn RESTful & HTTP Status Code

FieldController.java (Provider Service - Port 8081)

```java
@RestController
@RequestMapping("/api/fields")
public class FieldController {
    private final Map<String, FieldResponse> db = new ConcurrentHashMap<>();

    public FieldController() {
        // Dữ liệu mẫu tương đồng Alobo Catalog
        db.put("f-101", new FieldResponse(
            "f-101", "Sân bóng cỏ nhân tạo 7 người - Sân A",
            "FOOTBALL", new BigDecimal("350000"), "AVAILABLE"
        ));
    }

    @GetMapping("/{id}")
    public ResponseEntity<?> getFieldById(@PathVariable String id) {
        FieldResponse field = db.get(id);
        if (field == null) {
            // Trả về 404 Not Found nếu không tìm thấy ID
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(Map.of("status", 404, "error", "Sân '" + id + "' không tồn tại"));
        }
        // Trả về 200 OK kèm thông tin JSON
        return ResponseEntity.ok(field);
    }
}
```

DTO JAVA RECORD (IMMUTABLE DATA)

```java
public record FieldResponse(
    String id,
    String name,
    String sportType,
    BigDecimal pricePerHour,
    String status
) {}
```

NGHIỆP VỤ & NGUYÊN TẮC CỐT LÕI

- **Tính Idempotent (Bảo toàn trạng thái):** Phương thức `GET` không làm thay đổi trạng thái dữ liệu, an toàn cho việc gọi lại nhiều lần (Retry).
- **Chuẩn hóa Mã Lỗi HTTP:**
  - Trả `200 OK` khi tìm thấy bản ghi.
  - Trả `404 Not Found` khi ID không có trong Database. Tuyệt đối không trả về 200 kèm payload rỗng.
- **Tính tương thích cao:** Dữ liệu JSON được tuần tự hóa tự động qua Jackson, hỗ trợ mọi Consumer dù viết bằng bất cứ ngôn ngữ nào.

2.2 Xây Dựng Consumer (booking-service) Với RestClient

Tiếp nhận đặt sân POST /api/bookings, gọi sang Field Service & xử lý 200 OK / 404 Not Found

BookingController.java (Consumer - Port 8080)

```java
@RestController
@RequestMapping("/api/bookings")
public class BookingController {
    private final RestClient fieldClient;

    public BookingController(RestClient.Builder builder) {
        this.fieldClient = builder.baseUrl("http://localhost:8081").build();
    }

    @PostMapping
    public ResponseEntity<?> createBooking(@RequestBody BookingRequest req) {
        try {
            // 1. Gọi sang field-service kiểm tra sự tồn tại của fieldId
            FieldDto field = fieldClient.get()
                .uri("/api/fields/{id}", req.fieldId())
                .retrieve()
                .onStatus(HttpStatusCode::is4xxClientError, (request, response) -> {
                    throw new FieldNotFoundException("Sân '" + req.fieldId() + "' không tồn tại!");
                })
                .body(FieldDto.class);

            // 2. Nhận 200 OK: Tiếp tục lưu đơn đặt sân vào DB
            Booking saved = new Booking(UUID.randomUUID().toString(), req, field);
            return ResponseEntity.status(HttpStatus.CREATED).body(saved);

        } catch (FieldNotFoundException ex) {
            // 3. Nhận 404: Không tạo lịch, trả về lỗi báo sân không tồn tại
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(Map.of("error", ex.getMessage()));
        }
    }
}
```

REQUEST BODY TỪ CLIENT

```java
public record BookingRequest(
    String customerId,
    String fieldId,
    LocalDate bookingDate,
    LocalTime startTime,
    LocalTime endTime
) {}
```

TẠI SAO DÙNG RESTCLIENT TRONG SPRING BOOT 3?

- **Hiện đại & Đồng bộ (Fluent API):** Thay thế `RestTemplate` (đang ở trạng thái bảo trì), cú pháp tương tự WebClient nhưng chạy đồng bộ trực quan.
- **Xử lý Lỗi Tinh gọn (onStatus):** Bắt mã lỗi 4xx/5xx ngay trong chuỗi method chaining mà không cần viết các khối try-catch rườm rà.
- **Tự động ánh xạ JSON:** Sử dụng HttpMessageConverters để parse JSON trả về trực tiếp thành Java Record `FieldDto` an toàn kiểu (Type-safe).

2.3 Kịch Bản Kiểm Thử & Đánh Giá Kết Quả Thực Nghiệm

Kiểm chứng 2 kịch bản nghiệp vụ: Đặt sân hợp lệ (200 OK) và Đặt sân không tồn tại (404 Error)

TEST CASE 1: ĐẶT SÂN THÀNH CÔNG (200 OK)

PASS

Request Client gửi lên:

```
POST http://localhost:8080/api/bookings
{
  "customerId": "cust-889",
  "fieldId": "f-101",
  "bookingDate": "2026-10-10",
  "startTime": "18:00:00",
  "endTime": "20:00:00"
}
```

Kết quả phản hồi (201 Created):

```
{
  "bookingId": "BK-202610-A91F",
  "fieldId": "f-101",
  "fieldName": "Sân bóng cỏ nhân tạo 7 người - Sân A",
  "totalAmount": 700000.0,
  "status": "CONFIRMED"
}
```

✓ Booking Service xác thực sân thành công & lưu bản ghi vào Database.

TEST CASE 2: SÂN KHÔNG TỒN TẠI (404 NOT FOUND)

ERROR CAUGHT

Request Client gửi lên với ID không có:

```
POST http://localhost:8080/api/bookings
{
  "customerId": "cust-889",
  "fieldId": "f-999",  // ID giả lập không tồn tại
  "bookingDate": "2026-10-10",
  "startTime": "18:00:00",
  "endTime": "20:00:00"
}
```

Kết quả phản hồi (404 Not Found):

```
{
  "timestamp": "2026-10-05T14:02:15Z",
  "status": 404,
  "error": "Sân 'f-999' không tồn tại!"
}
```

✓ Hoàn toàn KHÔNG tạo lịch rác trong DB, bảo toàn tính toàn vẹn dữ liệu.

3.1 Viết Lại Consumer Bằng FastAPI (Python 3.12 + HTTPX)

Triển khai Asynchronous Consumer đồng nhất với module python_backend tại D:\hdvackend

booking_service.py (FastAPI + Pydantic + HTTPX)

```python
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel
import httpx, os

app = FastAPI(title="Booking Service (FastAPI)")
FIELD_SERVICE_URL = os.getenv("FIELD_SERVICE_URL", "http://localhost:8081")

class BookingRequest(BaseModel):
    customerId: str
    fieldId: str
    bookingDate: str
    startTime: str
    endTime: str

@app.post("/api/bookings", status_code=status.HTTP_201_CREATED)
async def create_booking(req: BookingRequest):
    # 1. Gọi sang field-service bằng httpx.AsyncClient (Timeout 3s)
    async with httpx.AsyncClient(timeout=3.0) as client:
        try:
            resp = await client.get(f"{FIELD_SERVICE_URL}/api/fields/{req.fieldId}")
        except httpx.ConnectError:
            raise HTTPException(status_code=503, detail="Field Service không phản hồi")

    # 2. Xử lý 404: Không tạo lịch, trả về lỗi sân không tồn tại
    if resp.status_code == 404:
        raise HTTPException(status_code=404, detail=f"Sân '{req.fieldId}' không tồn tại!")

    field = resp.json()
    # 3. Nhận 200 OK: Tiếp tục lưu đơn đặt sân và trả về thành công
    return {
        "bookingId": f"BK-PY-{os.urandom(4).hex().upper()}",
        "customerId": req.customerId, "fieldId": req.fieldId,
        "fieldName": field.get("name"), "status": "CONFIRMED"
    }
```

ƯU THẾ TUYỆT VỜI CỦA FASTAPI CONSUMER

- **Thực Tế Từ Dự Án:** Trong `D:\hdvackend\python_backend` đã có sẵn FastAPI, Uvicorn, và HTTPX, giúp triển khai tức thì không cần cài thêm công cụ mới.
- **AsyncIO & HTTPX Non-blocking:** Xử lý đồng thời hàng ngàn request I/O-bound với tài nguyên CPU và RAM tối thiểu.
- **Data Validation với Pydantic:** Dữ liệu được ép kiểu và kiểm tra tính hợp lệ tự động; tự sinh tài liệu tương tác **Swagger UI (/docs)**.

CƠ CHẾ ASYNC CONTEXT MANAGER

Việc dùng `async with httpx.AsyncClient()` đảm bảo kết nối TCP/HTTP pool được quản lý tự động, tự giải phóng socket ngay khi hoàn tất request hoặc khi gặp lỗi timeout.

3.2 Bảng So Sánh Toàn Diện: Spring Boot vs FastAPI

Đánh giá 5 tiêu chí kỹ thuật: Ngôn ngữ, HTTP Client, JSON, Khởi động, Mức độ phức tạp

Tiêu Chí Đánh Giá	Spring Boot 3 (Java 21)	FastAPI (Python 3.12)
1. Ngôn Ngữ & Mô Hình Luồng	Java: Kiểu dữ liệu tĩnh (Static Typing). Mô hình đa luồng JVM (Multi-threading), hỗ trợ *Virtual Threads (Project Loom)* xử lý tải lớn.	Python: Kiểu động kết hợp Type Hints. Mô hình đơn luồng dựa trên cơ chế *AsyncIO Event Loop (uvloop)* xử lý non-blocking I/O siêu mượt.
2. Cú Pháp HTTP Client	Sử dụng `RestClient` hoặc `WebClient`. Cú pháp Fluent API hướng đối tượng, kiểm tra kiểu chặt chẽ (Type-safe), xử lý lỗi qua `onStatus()`.	Sử dụng thư viện `httpx.AsyncClient` hoặc `aiohttp`. Cú pháp `async / await` kết hợp `async with` cực kỳ tinh gọn, dễ đọc.
3. Cấu Hình & Xử Lý JSON	Phụ thuộc vào `Jackson ObjectMapper`. Cần định nghĩa cấu trúc qua Java Record hoặc POJO Class; ánh xạ qua cơ chế Reflection.	Tự động hóa hoàn toàn nhờ `Pydantic BaseModel`. Tự động kiểm tra dữ liệu đầu vào (Validation) và serialize JSON mà không cần code boilerplate.
4. Thời Gian Khởi Động (Startup Time)	Trung bình **1.5 – 4.5 giây** (Do quá trình khởi tạo JVM, quét Component Scan, cấu hình IoC Container).	Khởi động tức thì: **0.2 – 0.5 giây**. Rất lý tưởng cho môi trường Container (Docker), Kubernetes Auto-scaling và Serverless.
5. Mức Độ Phức Tạp & Tiêu Tốn RAM	Mức độ phức tạp: **Trung bình – Cao**. Yêu cầu tuân thủ cấu trúc IoC, DI, Maven. Tiêu thụ RAM trung bình từ **150MB – 350MB**.	Mức độ phức tạp: **Thấp – Trung bình**. Cú pháp ngắn gọn, dễ tiếp cận, tích hợp sẵn Swagger UI. Tiêu thụ RAM thấp: **40MB – 80MB**.

KẾT LUẬN LỰA CHỌN KIẾN TRÚC

- Chọn **Spring Boot** cho các dịch vụ nghiệp vụ cốt lõi, giao dịch thanh toán (Payment Service) đòi hỏi tính nhất quán ACID và kiểm soát bảo mật đa tầng.
- Chọn **FastAPI (Python)** cho các dịch vụ Consumer, API Gateway, xử lý dữ liệu truyền thông đa phương tiện (Media Service) và các tác vụ AI/ML phân tích dữ liệu sân thể thao.

4.1 Xử Lý Lỗi Mạng: Bắt Lỗi Connection Refused

Bảo đảm tính bền vững (Resilience) khi Provider đột ngột ngừng hoạt động, không để sập ứng dụng

TÌNH HUỐNG LỖI MẠNG (DOWNSTREAM OUTAGE)

Kịch bản thực nghiệm: Tắt hoàn toàn `field-service` (port 8081). Sau đó gửi request đặt sân đến `booking-service` (port 8080).

```
org.springframework.web.client.ResourceAccessException: I/O error on GET request for "http://localhost:8081/api/fields/f-101": ConnectException: Connection refused: connect
```

- **Nếu KHÔNG xử lý ngoại lệ:** Lỗi kết nối sẽ văng thẳng ra ngoài luồng, ứng dụng trả về mã lỗi `500 Internal Server Error` kèm toàn bộ StackTrace nhạy cảm, gây hoang mang cho người dùng và vi phạm an toàn thông tin.
- **Mục tiêu thiết kế:** Bắt gọn ngoại lệ mạng, ghi log cảnh báo và trả về phản hồi lịch sự `503 Service Unavailable`.

GlobalExceptionHandler.java (Xử lý tập trung)

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceAccessException.class)
    public ResponseEntity<?> handleNetworkDown(ResourceAccessException ex) {
        // Bắt lỗi kết nối mạng (Connection refused)
        return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE)
            .body(Map.of(
                "timestamp", Instant.now().toString(),
                "status", 503,
                "error", "Service Unavailable",
                "message", "Hệ thống quản lý sân bãi đang bảo trì. Vui lòng thử lại sau!"
            ));
    }
}
```

KẾT QUẢ ĐẠT ĐƯỢC:

✓ Ứng dụng booking-service **vẫn chạy ổn định 100%**, không crash.

✓ Client nhận được mã 503 cùng chỉ dẫn rõ ràng thay vì màn hình trắng.

4.2 Cấu Hình Timeout (3s) & Xử Lý 504 Gateway Timeout

Giả lập Provider phản hồi trễ (5–10s) và ngắt kết nối tự động để tránh cạn kiệt tài nguyên

Cấu hình RestClient Timeout 3 Giây (Spring Boot)

```java
@Configuration
public class RestClientConfig {
    @Bean
    public RestClient fieldRestClient() {
        // Cấu hình thời gian chờ kết nối và chờ đọc dữ liệu
        ClientHttpRequestFactorySettings settings =
            ClientHttpRequestFactorySettings.DEFAULTS
                .withConnectTimeout(Duration.ofSeconds(2))
                .withReadTimeout(Duration.ofSeconds(3)); // Timeout 3s

        return RestClient.builder()
            .requestFactory(ClientHttpRequestFactories.get(settings))
            .baseUrl("http://localhost:8081")
            .build();
    }
}
```

Giả lập trễ tại Field Service:

```java
@GetMapping("/{id}")
public ResponseEntity<?> getField(...) throws InterruptedException {
    Thread.sleep(8000); // Giả lập nghẽn mạng hoặc database trễ 8 giây
    return ResponseEntity.ok(field);
}
```

XỬ LÝ NGOẠI LỆ 504 GATEWAY TIMEOUT

```java
@ExceptionHandler(SocketTimeoutException.class)
public ResponseEntity<?> handleTimeout() {
    return ResponseEntity.status(HttpStatus.GATEWAY_TIMEOUT)
        .body(Map.of(
            "status", 504,
            "error", "Gateway Timeout",
            "message", "Field Service phản hồi vượt quá 3s. Hủy giao dịch!"
        ));
}
```

Ý NGHĨA QUAN TRỌNG CỦA TIMEOUT TRONG MICROSERVICES

- **Chống Thắt Cổ Chai (Avoid Thread Starvation):** Nếu không có timeout, mỗi request của khách sẽ chiếm giữ 1 worker thread trong 30–60 giây. Chỉ cần 200 khách gọi là toàn bộ server Tomcat cạn kiệt luồng và sập hoàn toàn.
- **Nguyên tắc Fail-Fast:** Phát hiện lỗi sớm và ngắt ngay lập tức giúp hệ thống nhanh chóng giải phóng RAM/CPU để phục vụ các yêu cầu khác.

4.3 Giao Tiếp Bất Đồng Bộ Với Hàng Đợi RabbitMQ

Thiết kế mô hình Event-Driven giải phóng kết nối cho tác vụ thông báo đặt sân mới

VẤN ĐỀ CỦA GỌI ĐỒNG BỘ KHI GỬI THÔNG BÁO

Sau khi lưu đơn đặt sân thành công, nếu Booking Service gọi HTTP trực tiếp đến Notification Service:

- Gửi SMS/Email/Push thường mất từ **1 đến 3 giây** do phụ thuộc vào bên thứ 3 (Twilio, Firebase, SendGrid).
- Nếu hệ thống thông báo quá tải hoặc bị lỗi, đơn đặt sân của khách hàng cũng sẽ bị báo lỗi theo!

GIẢI PHÁP EVENT-DRIVEN (RABBITMQ):

Booking Service chỉ việc **bắn sự kiện BookingCreated** vào hàng đợi RabbitMQ rồi trả ngay kết quả cho khách (thời gian dưới 10ms). Notification Service sẽ tự động lấy tin nhắn xử lý ngầm (Background Worker).

SƠ ĐỒ LUỒNG TIN NHẮN (TOPOLOGY RABBITMQ)



Exchange: `booking.exchange` | Queue: `notification.booking.queue` | RoutingKey: `booking.created`

4.4 Triển Khai RabbitMQ: Producer & Consumer Chi Tiết

Gói tin BookingCreatedEvent và xử lý in console log tại Notification Service

BookingProducer.java (booking-service)

```java
@Service
public class BookingProducer {
    private final RabbitTemplate rabbitTemplate;

    public BookingProducer(RabbitTemplate rabbitTemplate) {
        this.rabbitTemplate = rabbitTemplate;
    }

    public void publishBookingCreated(Booking booking) {
        BookingCreatedEvent event = new BookingCreatedEvent(
            booking.getBookingId(),
            booking.getFieldId(),
            booking.getCustomerId(),
            Instant.now()
        );
        // Đẩy sự kiện vào RabbitMQ
        rabbitTemplate.convertAndSend(
            "booking.exchange",
            "booking.created",
            event
        );
        System.out.println("[PRODUCER] Đã bắn sự kiện đặt sân: " + booking.getBookingId());
    }
}
```

NotificationConsumer.java (notification-service)

```java
@Component
public class NotificationConsumer {

    @RabbitListener(queues = "notification.booking.queue")
    public void handleBookingCreated(BookingCreatedEvent event) {
        System.out.println("============================================
        System.out.println("📢 [NOTIFICATION SERVICE] NHẬN ĐƯỢC ĐƠN ĐẶT S
        System.out.println(" ➔ Booking ID : " + event.bookingId());
        System.out.println(" ➔ Field ID   : " + event.fieldId());
        System.out.println(" ➔ Customer ID: " + event.customerId());
        System.out.println(" ➔ Thời gian  : " + event.timestamp());
        System.out.println(" Đang gửi Email & Push Notification tới khách
        System.out.println("============================================
    }
}
```

TÍNH BẢO ĐẢM TIN NHẮN (MESSAGE DURABILITY):
Khi Notification Service tắt bảo trì, tin nhắn vẫn được lưu an toàn trong Queue của RabbitMQ. N
service sẽ tự động đọc và gửi bù toàn bộ thông báo mà không làm mất bất kỳ đơn nào!

5.1 Thử Thách 1: Tránh Hard-Code Bằng Cấu Hình Động

Externalized Configuration chuẩn 12-Factor App trong application.yml và Docker

application.yml (Externalized Configuration)

```yaml
server:
  port: 8080

services:
  field-service:
    # Nạp động từ biến môi trường FIELD_SERVICE_URL
    # Mặc định localhost:8081 nếu không khai báo
    url: ${FIELD_SERVICE_URL:http://localhost:8081}
    connect-timeout-ms: 2000
    read-timeout-ms: 3000

spring:
  rabbitmq:
    host: ${RABBITMQ_HOST:localhost}
    port: 5672
```

FieldServiceProperties.java (Type-safe Config)

```java
@ConfigurationProperties(prefix = "services.field-service")
public record FieldServiceProperties(
    String url,
    int connectTimeoutMs,
    int readTimeoutMs
) {}
```

ƯU THẾ TUYỆT ĐỐI KHI TRIỂN KHAI PRODUCTION

- **Không cần Rebuild mã nguồn:** Khi chuyển từ máy cá nhân lên Docker Swarm hay Kubernetes Cluster, chỉ cần truyền biến môi trường:
  `docker run -e FIELD_SERVICE_URL=http://prod-field:8081 ...`
- **Bảo mật & Tách biệt:** Không để lộ các địa chỉ mạng nội bộ, thông tin đăng nhập trong repository Git mã nguồn.

5.2 Thử Thách 2: Cơ Chế Tự Động Thử Lại (Retry) & Fallback

Triển khai Resilience Pattern tự phục hồi trước lỗi mạng chập chờn (Transient Faults)

FieldClientService.java (Tích hợp Spring Retry)

```java
@Service
public class FieldClientService {
    private final RestClient restClient;

    @Retryable(
        retryFor = { ResourceAccessException.class }, // Lỗi mạng tạm thời
        maxAttempts = 3,                              // Thử tối đa 3 lần
        backoff = @Backoff(delay = 1000, multiplier = 1.5) // Giãn cách lũy tiến
    )
    public FieldDto getFieldDetails(String fieldId) {
        System.out.println("Đang kết nối Field Service (lần gọi)...");
        return restClient.get()
            .uri("/api/fields/{id}", fieldId)
            .retrieve()
            .body(FieldDto.class);
    }

    @Recover
    public FieldDto fallbackField(ResourceAccessException ex, String id) {
        System.err.println("Đã thử 3 lần thất bại. Kích hoạt Fallback!");
        throw new ServiceFallbackException(
            "Hệ thống kiểm tra sân bận. Đơn tạm thời chuyển vào hàng chờ duyệt!"
        );
    }
}
```

CƠ CHẾ HOẠT ĐỘNG CỦA RETRY & BACKOFF

- **Lần 1:** Gặp lỗi chập chờn gói tin (Network Glitch). Service không báo lỗi ngay mà dừng **1000ms**.
- **Lần 2:** Tiếp tục thử lại lần 2. Nếu vẫn lỗi, tăng thời gian chờ lên **1500ms** (Multiplier 1.5).
- **Lần 3:** Thử lại lần cuối. Nếu Provider đã hồi phục, request hoàn thành mà người dùng không hề biết đã có lỗi xảy ra!

KỊCH BẢN PHỤC HỒI FALLBACK

Khi cả 3 lần thử đều thất bại, hàm `@Recover` tiếp quản để đưa ra giải pháp thay thế: chuyển giao dịch vào trạng thái *PENDING_APPROVAL*, hoặc trả về phản hồi fallback thân thiện thay vì làm sập cả hệ thống.

5.3 Đánh Giá Cuối Bài: Giải Đáp Toàn Diện 5 Câu Hỏi Exit Ticket

Tổng hợp các câu hỏi trọng tâm chốt kiến thức môn học kiến trúc Microservices

Câu 1: REST API và Spring Boot có phải là một khái niệm không?

KHÔNG. REST API là một *phong cách kiến trúc phần mềm* (Architectural Style) định nghĩa các nguyên tắc truyền thông qua giao thức HTTP (stateless, tài nguyên định danh qua URI). Trong khi đó, Spring Boot là một *Framework / Nền tảng phát triển ứng dụng Java*. Lập trình viên có thể dùng Spring Boot để tạo ra REST API, hoặc tạo ứng dụng console, worker, SOAP web service...

Câu 2: Trong bài toán trên, service nào là Provider, service nào là Consumer?

Field Service là Provider (nhà cung cấp dữ liệu sân bãi qua API `GET /api/fields/{id}`). **Booking Service là Consumer** (bên tiêu thụ API của Field Service để xác thực đơn). Trong trường hợp RabbitMQ: Booking Service là Producer (bắn sự kiện), còn Notification Service là Consumer (lắng nghe sự kiện).

Câu 3: Điều gì xảy ra với việc đặt sân nếu field-service ngừng hoạt động?

Việc tạo lịch đặt sân mới sẽ **bị gián đoạn ngay lập tức** vì Booking Service không thể xác thực sân. Nếu hệ thống có xử lý lỗi & timeout, nó sẽ từ chối đơn một cách an toàn (trả về HTTP 503 hoặc chuyển vào hàng chờ xét duyệt) để tránh tình trạng đặt nhầm sân hư hoặc đặt chồng lấn lịch (Overbooking).

Câu 4: Phân biệt giao tiếp đồng bộ (HTTP) và bất đồng bộ (RabbitMQ)?

Đồng bộ (HTTP/REST): Consumer gửi request và *bắt buộc phải chờ* Provider phản hồi mới đi tiếp (Chặt chẽ, tốn thread, phụ thuộc thời gian thực).
Bất đồng bộ (RabbitMQ): Producer đẩy thông điệp vào hàng đợi rồi *trả kết quả ngay lập tức*; Consumer tự lấy xử lý ngầm (Lỏng lẻo, không nghẽn luồng, bền vững).

Câu 5: Khi nào trong bài toán đặt sân nên dùng RabbitMQ thay vì gọi REST trực tiếp?

Nên dùng RabbitMQ cho các tác vụ **hậu kỳ (Side-effects / Background Jobs)** không cần trả kết quả tức thì cho người dùng: Gửi Email/SMS xác nhận, gửi Push Notification cho chủ sân, tích điểm thưởng khách hàng thân thiết, cập nhật bảng thống kê doanh thu (Analytics), hoặc phát sự kiện realtime qua SSE/Socket (như trong `D:\hdvackendpps ealtime`).

TỔNG KẾT CHUYÊN ĐỀ KẾT THÚC

Tổng Kết Kiến Trúc & Khuyến Nghị Vận Hành

Các nguyên tắc vàng rút ra khi xây dựng hệ thống phân tán chịu lỗi cao

1. GIAO TIẾP HYBRID

Kết hợp hài hòa giữa **REST API Đồng bộ** cho các luồng dữ liệu đòi hỏi tính nhất quán tức thì (kiểm tra tồn tại sân, tính tiền) và **RabbitMQ Bất đồng bộ** cho các tác vụ phụ trợ (gửi thông báo, phân tích).

2. PHÒNG VỆ CHỦ ĐỘNG

Luôn đặt **Timeout** (2–3 giây) cho mọi kết nối mạng giữa các service. Bổ sung cơ chế **Retry với Backoff** và khối xử lý **Global Exception Handler** để ngăn sập dây chuyền (Cascading Outage).

3. CẤU HÌNH & VẬN HÀNH

Tuyệt đối tránh ghi cứng URL. Sử dụng **Environment Variables** và file cấu hình động theo chuẩn Cloud-Native / Docker để dễ dàng scale-up và bảo trì lâu dài.

CẢM ƠN THẦY CÔ & CÁC BẠN ĐÃ LẮNG NGHE!

Hệ thống mẫu đã được kiểm thử đồng bộ với kiến trúc dự án thực tế tại thư mục `D:\hdvackend`.

MÃ NGUỒN: ALOBO BACKEND | TRẠNG THÁI: HOÀN THÀNH TOÀN BỘ 5 BÀI | HỎI & ĐÁP (Q&A)